

Reviewer Pack — Trivial-Isotype Mass Gap Separates Perm from Padded Det

Entry 2d98562b9959 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 23:19:48 UTC

Trivial-Isotype Mass Gap Separates Perm from Padded Det

Entry ID: 2d98562b9959 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 23:19:48 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl / Specht-isotype decomposition of the regular representation of S_n applied to the permutation-diagonal coefficient vector of homogeneous polynomials — an algebraic-combinatorial structural invariant in the Stembridge-Greene-James-Kerber tradition; an arXiv search ‘Specht isotype’ AND (‘determinantal complexity’ OR ‘orbit closure’ OR ‘padded determinant’) returns 0 direct hits, with <5 adjacent papers (none mapping Specht-isotype mass to GCT obstructions of perm_n). **Field B** (complexity object): Determinantal complexity $\text{dc}(\text{perm}_n)$ in the Mulmuley-Sohoni GCT_DET_PERM regime — the smallest m such that perm_n lies in the $GL_{\{n^2\}}$ -orbit closure of $\square(y)^{\{n-m\}} \cdot \det_m(L(y))$ where L is an $m \times m$ matrix of linear forms in $y = \{y_{\{ij\}}\}_{1 \leq i, j \leq n}$, the canonical Landsberg 2017 / Mignon-Ressayre target for super-polynomial $\det$ -vs- perm separation.

Statement:

For homogeneous $f \in R[y_{\{ij\}}]_{\{1 \leq i, j \leq n\}}$ of degree n , let $v(f) \in R^{\{S_n\}}$ be the permutation-diagonal vector $v_{\sigma}(f) = [\prod_i y_{\{i, \sigma(i)\}}] f$, and set $\rho(f) = (\sum_{\sigma} v_{\sigma}(f)^2) / (n! \cdot \sum_{\sigma} v_{\sigma}(f)^2)$ when $v(f) \neq 0$ (the squared cosine of $v(f)$ with the trivial-isotype direction of the regular S_n -representation). Then (i) $\rho(\text{perm}_n) = 1$ (saturation), and (ii) for every $m \times m$ matrix L of linear forms $L_{\{ab\}}(y)$ and every linear form $\square(y)$ in the n^2 variables, with $2 \leq m \leq n-1$, the padded form $f_{\{L, \square\}} = \square(y)^{\{n-m\}} \cdot \det_m(L(y))$ satisfies $\rho(f_{\{L, \square\}}) \leq 1 - 1/n$. A single pair $(L, \square)$ achieving $\rho(f_{\{L, \square\}}) > 1 - 1/n$ refutes the conjecture.

Rationale (proposer’s reasoning):

Perm_n uniquely saturates the trivial-isotype of $R^{\{S_n\}}$ because its permutation-diagonal coefficient vector is the all-ones direction; $\det_n$ analogously saturates the sign-isotype. Padded $\det_m$ factors through an $m \times m$ antisymmetric tensor (sign-isotype of $S_m \hookrightarrow S_n$) tensored with a symmetric $\square^{\{n-m\}}$ factor, and by a Pieri/branching argument the induced direction in $R^{\{S_n\}}$ cannot reach the trivial saturation when $m < n$. Because ρ is computed from permutation-diagonal coefficients via finite Gaussian-elimination steps (no truth-table evaluation), the invariant is RELATIONAL on the polynomial structure and shielded from Razborov-Rudich (it is not a poly-time-computable property of a truth table) and from algebrization (the underlying claim is a representation-theoretic projection-saturation, not a polynomial identity over a ring extension).

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dbb6b816a6d7f5be

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each (n,m) with $n \in \{4,5,6\}$, $m \in \{2, \lfloor n/2 \rfloor, n-1\}$, draw 30 Gaussian seeds and compute $\rho(f_{\{L, \square\}})$. SUPPORTED iff $\max \rho$ over all 90 seeds $< 1-1/n$ at its respective n , AND sanity check $\rho(\text{perm}_n)=1$ to tol $1e-9$. FALSIFIED by any seed with $\rho \geq 1-1/n$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Specht isotype permanent determinantal complexity orbit closure - trivial representation S_n permanent padded determinant GCT obstruction - symmetric group isotypic decomposition diagonal coefficients permanent versus determinant

Top relevant hits considered: - [http://arxiv.org/abs/1306.1810v6] Matrix orbit closures - [http://arxiv.org/abs/2202.13016v1] Bounds on Determinantal Complexity of Two Types of Generalized Permanents - [http://arxiv.org/abs/1007.3804v4] Symmetric Determinantal Representation of Formulas and Weakly Skew Circuits - [http://arxiv.org/abs/1503.00822v2] Product Ranks of the 3×3 Determinant and Permanent - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/1005.0140v4] Representations of hom-Lie algebras - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from fractions import Fraction

def matrix_mult(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
```

```

result = [[0 for _ in range(m)] for _ in range(n)]
for i in range(n):
    for j in range(m):
        for k in range(p):
            result[i][j] += A[i][k] * B[k][j]
return result

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_inverse(A):
    n = len(A)
    I = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(n)]
    for col in range(n):
        if A[col][col] == 0:
            for row in range(col + 1, n):
                if A[row][col] != 0:
                    A[col], A[row] = A[row], A[col]
                    I[col], I[row] = I[row], I[col]
                    break
        pivot = A[col][col]
        for j in range(n):
            A[col][j] = Fraction(A[col][j], pivot)
            I[col][j] = Fraction(I[col][j], pivot)
        for i in range(n):
            if i != col and A[i][col] != 0:
                factor = A[i][col]
                for j in range(n):
                    A[i][j] -= factor * A[col][j]
                    I[i][j] -= factor * I[col][j]

    return I

def det(A):
    n = len(A)
    det_val = Fraction(1)
    for col in range(n):
        if A[col][col] == 0:
            for row in range(col + 1, n):
                if A[row][col] != 0:
                    A[col], A[row] = A[row], A[col]
                    det_val *= -1
                    break
        if A[col][col] == 0:
            return Fraction(0)
        det_val *= A[col][col]
        for row in range(col + 1, n):
            factor = Fraction(A[row][col], A[col][col])
            for c in range(col, n):
                A[row][c] -= factor * A[col][c]
    return det_val

def generate_permutation(n):
    perm = list(range(n))
    random.shuffle(perm)
    return perm

```

```

def generate_random_matrix(n, m, seed):
    random.seed(seed)
    return [[random.gauss(0, 1) for _ in range(n)] for _ in range(m)]

def generate_random_vector(n, seed):
    random.seed(seed)
    return [random.gauss(0, 1) for _ in range(n)]

def compute_v_sigma(f_L_ell, sigma, n, m, L, ell):
    v_sigma = 0
    for psi in itertools.permutations(range(n), m):
        M_sigma_psi = [[L[a][b][psi[a] * n + sigma[psi[a]] for b in range(m)] for a in range(m)]
        det_M = det(M_sigma_psi)
        product_ell = 1
        for p in range(n):
            if p not in psi:
                product_ell *= ell[p * n + sigma[p]]
        v_sigma += det_M * product_ell
    return v_sigma * math.factorial(n - m)

def compute_rho(f_L_ell, n, m, L, ell):
    v = [compute_v_sigma(f_L_ell, sigma, n, m, L, ell) for sigma in itertools.permutations(range(n), m)]
    sum_v = sum(v)
    sum_v_sq = sum(x * x for x in v)
    if sum_v_sq == 0:
        return 0
    return (sum_v * sum_v) / (math.factorial(n) * sum_v_sq)

def run_trial(seed):
    n_values = [4, 5, 6]
    m_values = [[2], [n//2], [n-1]]
    max_rho = 0
    counterexample = ""
    instances_tested = 0

    for n in n_values:
        for m in m_values:
            random.seed(seed)
            L = [[generate_random_vector(n * n, seed + i + j) for j in range(n)] for i in range(m)]
            ell = generate_random_vector(n * n, seed + m)
            f_L_ell = lambda y: sum(ell[i] * y[i] for i in range(n * n)) ** (n - m) * det([[L[a][b][i] for b in range(m)] for a in range(m)])
            rho = compute_rho(f_L_ell, n, m, L, ell)
            instances_tested += 1
            if rho > max_rho:
                max_rho = rho
            if rho >= 1 - 1/n:
                counterexample = f"rho={rho} >= 1-1/{n} for n={n}, m={m}"
                break
        if counterexample:
            break

    conjecture_holds = max_rho < 1 - 1/n_values[-1]
    return {
        "metric_name": "max_rho",
        "metric_value": max_rho,
        "instances_tested": instances_tested,
    }

```

```

        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [random.randint(1, 1000)]
    metric_values = []
    conjecture_holds_counts = 0
    counterexamples = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        metric_values.append(result["metric_value"])
        if result["conjecture_holds"]:
            conjecture_holds_counts += 1
        if result["counterexample"]:
            counterexamples.append((seed, result["counterexample"]))

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = conjecture_holds_counts / len(seeds)

    if counterexamples:
        seed, counterexample = counterexamples[0]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seed}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ba97aec.py", line 144, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ba97aec.py", line 107, in run_trial

```

    m_values = [[2], [n//2], [n-1]]
    ^

```

UnboundLocalError: cannot access local variable 'n' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an UnboundLocalError before producing any p values, so neither the support nor the falsification condition could be evaluated. | next: Fix the scoping bug (define n before constructing m_values, e.g., inside the n-loop) and rerun the 90-seed sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	306004
2	preregistration	claude_max	opus	0	0	9798
3	novelty	claude_max	opus	0	0	3865
4	novelty	claude_max	opus	0	0	10910
5	test_gen	mistral	codestral-latest	0	0	310644
6	test_gen	mistral	codestral-latest	0	0	312075
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	294699
8	test_gen	mistral	codestral-latest	0	0	311869
9	judge	claude_max	opus	0	0	4858

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1564722 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in research/audit/2d98562b9959.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2d98562b9959.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2d98562b9959.tar.gz (if generated)